Teachable Machine AI

Full-Stack Image Classification Application

Project Documentation

Streamlit UI + FastAPI ML Engine + Transfer Learning + Docker Compose

Prepared For	Bootcamp Challenge / Project Evaluation
Prepared By	Usaid Ahmed
Main Goal	Build a real-world Teachable Machine-style AI app
Deliverable	Working full-stack AI application with documentation

1. Executive Summary

Teachable Machine AI is a full-stack image classification application inspired by Google Teachable Machine. The project allows users to create custom image classes, upload or capture training samples, train a custom model, and test predictions using either file upload or webcam input. The system is built as a real-world application rather than a notebook-only experiment.

The frontend is built with Streamlit and the backend is built with FastAPI. The machine learning engine uses transfer learning: MobileNetV3 acts as a pre-trained image feature extractor and Logistic Regression acts as a fast custom classifier trained on user-provided classes.

2. Project Objectives

- Move from notebook-based ML to a working full-stack AI application.
- Allow users to define custom image classes dynamically.
- Support both file upload and webcam-based sample collection.
- Train a transfer learning image classifier from user samples.
- Predict new images using file upload or webcam preview.
- Display prediction confidence scores for all trained classes.
- Keep user data isolated through session-based temporary datasets.
- Package the complete system using Docker Compose for easy execution.

3. Key Features

Feature	Description
Dynamic Class Cards	Users can create, rename, disable, enable, and delete custom classes.
File Upload Samples	Users can upload multiple images per class.
Webcam Sample Capture	Users can record samples from webcam and save them to the current class.
Transfer Learning Training	MobileNetV3 extracts image features and Logistic Regression trains the final classifier.
File Prediction	Users can upload a new image and predict its class.
Webcam Prediction	Users can capture the latest webcam frame and predict it.
Confidence Bars	The frontend displays class-wise prediction probabilities.
Export Model	Users can download the trained model package as a .pkl file.
Reset Project	Users can clear the current session and start again.
Docker Compose	Backend and frontend can run together with one command.

4. Technology Stack

Layer	Technologies
Frontend	Streamlit, streamlit-webrtc, OpenCV, Pillow, Requests
Backend	FastAPI, Uvicorn, Python Multipart
Machine Learning	PyTorch, Torchvision, MobileNetV3, Scikit-learn, Joblib, NumPy
Data Handling	Session-based temporary folders for datasets and models
DevOps	Docker, Docker Compose

5. System Architecture

The application is separated into two main services: a Streamlit frontend and a FastAPI backend. The frontend handles user interaction, while the backend handles file storage, model training, prediction, and model export.

```
User Browser
   │
   ▼
Streamlit Frontend (port 8501)
   │  HTTP requests
   ▼
FastAPI Backend (port 8000)
   │
   ├── Session Dataset Storage
   ├── MobileNetV3 Feature Extraction
   ├── Logistic Regression Training
   ├── Prediction API
   └── Model Export
```

6. Application Workflow

1. The user opens the Streamlit application.
2. The frontend creates a unique session ID for the user.
3. The user creates or edits class names.
4. The user uploads images or records webcam samples for each class.
5. The frontend sends samples to the FastAPI backend with the session ID.
6. The backend saves samples in a session-specific dataset folder.
7. The user clicks Train Model.
8. The backend extracts MobileNetV3 features and trains Logistic Regression.
9. The trained model is saved inside the current session model folder.
10. The user predicts new images from file upload or webcam frame.
11. The frontend displays predicted class and confidence bars.
12. The user can export the model or reset the project.

7. Machine Learning Design

7.1 Why Transfer Learning?

Training a full deep learning model from scratch requires a large dataset, powerful hardware, and long training time. This project uses transfer learning to make training fast and practical. MobileNetV3 already understands general visual features such as edges, shapes, colors, and object patterns. The app reuses those features and trains a lightweight classifier on top of them.

7.2 Training Pipeline

```
Training Images
   ->
Open image with Pillow
   ->
Convert to RGB
   ->
Resize to 224x224
```

```
   ->
Normalize with ImageNet mean/std
   ->
Extract features using MobileNetV3
   ->
Encode class labels
   ->
Train Logistic Regression
   ->
Calculate accuracy
   ->
Save model.pkl
```

7.3 Prediction Pipeline

```
New Image / Webcam Frame
   ->
Same preprocessing as training
   ->
MobileNetV3 feature extraction
   ->
Logistic Regression prediction
   ->
Class probabilities
   ->
Frontend confidence bars
```

8. Backend API Documentation

Method	Endpoint	Purpose
GET	/	Root API message.
GET	/health	Checks whether the backend is running.
POST	/upload-sample	Uploads class image samples. Requires session_id, class_name, and files.
POST	/train	Trains the model for the current session. Requires session_id.
POST	/predict	Predicts one uploaded image. Requires session_id and file.
GET	/dataset-summary	Returns classes and image counts for the current session.
DELETE	/delete-class	Deletes a class folder from the current session.
GET	/export-model	Downloads the trained model.pkl for the session.
DELETE	/reset-session	Deletes current session data.
POST	/cleanup-old-sessions	Deletes old session folders based on TTL.

9. Session-Based Data Handling

The app does not store all users' data in one shared global dataset folder. Instead, each user gets a unique session ID. Samples and models are saved in session-specific folders.

```
backend/sessions/<session_id>/dataset/<class_name>/
backend/sessions/<session_id>/models/model.pkl
```

When the user refreshes the frontend, the UI starts a new clean session. The old backend session folder is no longer used by that user and is removed later by the cleanup system according to SESSION_TTL_MINUTES. Clicking Reset Project deletes the current session immediately.

10. Frontend UI Design

- Class cards are inspired by Google Teachable Machine.
- Each card has editable class title, webcam button, upload button, and sample gallery.
- The Training panel validates classes before training.
- The Preview panel supports file and webcam prediction.
- Confidence bars show probabilities for every trained class.
- Backend status appears in the topbar.
- Reset Project clears the current workspace.
- Export Model downloads the trained model package.

11. Docker Setup

Docker is used to make the project portable. Instead of manually creating environments and installing dependencies, Docker Compose starts both backend and frontend together.

11.1 Docker Services

- backend: FastAPI service running on port 8000.
- frontend: Streamlit service running on port 8501.
- backend_sessions volume: stores temporary session files inside Docker.

11.2 Run With Docker Compose

```
docker compose up --build
```

Open the application at: http://localhost:8501

Open backend API documentation at: http://localhost:8000/docs

12. Manual Local Setup

12.1 Backend

```
cd backend
python -m venv venv
venv\Scriptsctivate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

12.2 Frontend

```
cd frontend
python -m venv venv
venv\Scriptsctivate
pip install -r requirements.txt
streamlit run app.py
```

14. Common Issues and Fixes

Issue	Fix
Backend Offline	Make sure FastAPI is running at http://127.0.0.1:8000 or use docker compose up --build.
Webcam Not Working	Use Chrome or Edge, open http://localhost:8501, and allow camera permission.
Docker Shows Old Code	Run docker compose down, then docker compose up --build. If needed, use docker compose build --no-cache.
Training Fails	Make sure at least two enabled classes have uploaded or webcam samples.
Prediction Fails	Make sure the model is trained before using prediction.

15. Future Improvements

- Continuous live webcam prediction every few seconds.
- Backend model caching for faster inference.
- User authentication and project saving.
- Cloud storage for datasets and trained models.
- Export to ONNX or TensorFlow format.
- Advanced model settings in the Training panel.
- Dataset quality scoring and class imbalance warnings.
- Deployment to a cloud platform.

17. Conclusion

This project successfully implements a real-world full-stack AI application with dynamic data ingestion, transfer learning, model training, prediction, webcam integration, session-based data handling, model export, reset functionality, and Docker-based deployment. It demonstrates both practical AI engineering and modern full-stack application development.